WILLMA APK

Security Analysis Report

Application: WILLMA (com.willma.client) v1.10.1

Platform: Android (React Native / Expo)

Analysis Date: June 1, 2026

Classification: Confidential

Executive Summary

A comprehensive security analysis of the WILLMA Android application (v1.10.1) has revealed **37 security findings** across multiple categories, including **6 CRITICAL**, **8 HIGH**, **12 MEDIUM**, **7 LOW**, and **4 INFO** severity issues. The most severe vulnerabilities involve hardcoded LLM API keys (OpenAI, Anthropic, DeepSeek, Gemini) embedded in plaintext within the app configuration, an SSL hostname verification bypass that accepts all certificates, and the Facebook client token exposed in resource files. These vulnerabilities could allow attackers to consume API credits at the app owner's expense, intercept encrypted communications, and impersonate the application on social platforms. Immediate remediation is strongly recommended before publishing to Google Play Store.

Risk Category	Count	Status
CRITICAL	6	MUST FIX BEFORE LAUNCH
HIGH	8	SHOULD FIX BEFORE LAUNCH
MEDIUM	12	RECOMMENDED TO FIX
LOW	7	LOW PRIORITY
INFO	4	INFORMATIONAL

Table of Contents

1. Application Overview

Property	Value
Package Name	com.willma.client
Application Name	WILLMA
Version	1.10.1 (versionCode: 21)
Min SDK	23 (Android 6.0 Marshmallow)
Target SDK	35 (Android 15)
Framework	React Native / Expo
Base APK Size	82 MB (Total XAPK: 119 MB)
Native Libraries	armeabi_v7a
Expo Project ID	0c01da2c-5409-459b-a106-98322e3d7076
Firebase Project	willma-prod
GraphQL API	https://app.willma.life/api/graphql

The WILLMA application is a React Native-based Android app built with the Expo framework. It integrates multiple AI provider APIs (OpenAI, Anthropic, DeepSeek, and Google Gemini) to provide conversational AI capabilities. The app also incorporates Facebook SDK for authentication, Firebase for messaging and analytics, Microsoft Clarity for session tracking, and Google ML Kit for barcode scanning. The application uses a GraphQL backend API hosted at app.willma.life. The app requests a significant number of permissions, including location access, camera, audio recording, calendar read/write, and phone state access, which raises concerns about the principle of least privilege.

2. Hardcoded Secrets and Credential Leak

The most critical finding in this analysis is the presence of multiple LLM API keys hardcoded in plaintext within the application's configuration file. The file **resources/assets/app.config** contains a JSON configuration block that embeds API keys for four different AI service providers. This means that anyone who extracts the APK (which is trivially done using publicly available tools) can retrieve these keys and use them to make API calls at the application owner's expense. The potential financial impact is significant, as LLM API calls can accumulate costs rapidly, especially if automated scripts are used to exhaust the keys.

2.1 LLM API Keys (CRITICAL)

Provider	Key (Truncated)	Impact
OpenAI	sk-proj-OsMIWUQ...G3EA	Full API access - text, images, massive cost risk
Anthropic	sk-ant-api03-Ceoak...WwAA	Full Claude API access - active provider
DeepSeek	sk-f4e060a5b624...2ff5	Full DeepSeek API access

Gemini	AIzaSyDaQPVTbMztW...ITE	Full Google Gemini API access

The Anthropic key is particularly valuable as the app configuration indicates **AI_PROVIDER=anthropic**, meaning this is the primary active AI provider. All four keys appear to be billing-enabled production keys with no apparent rate limiting or usage restrictions configured at the provider level. An attacker extracting the APK could automate API calls to any of these services, potentially incurring thousands of dollars in costs within hours. Additionally, the keys could be used to access any functionality available through the respective APIs, including generating content, accessing model fine-tuning endpoints, or extracting training data.

2.2 Other Hardcoded Credentials (HIGH)

Credential	Location	Value (Truncated)
Facebook Client Token	strings.xml	4b0f881668cbab35f07d4a6ed6ba00cb
Facebook App ID	strings.xml	1241712067901568
Google API Key	strings.xml	AIzaSyBfe2EJht-OaFnbfSNzXzCL96yMG3AXa7c
Ramadan API Key	app.config	NmQZJdz5skBXh9LPid6sgvDArJEgQKz3qxWtpI9mNn1brVxR
Sentry DSN	app.config	https://184ac66dc585...@o4509140680572928...
Firebase Project ID	strings.xml	willma-prod
Google Storage Bucket	strings.xml	willma-prod.firebasestorage.app
License RSA Public Key	LicenseClient.java	MIIBIjANBgkqhkiG9w0BAQEF...

The Facebook Client Token is particularly concerning as it allows API calls to be made on behalf of the Facebook application integration. Combined with the Facebook App ID, an attacker could impersonate the WILLMA app in Facebook API interactions, potentially accessing user data or performing actions as the app. The Google API key, if unrestricted, could be used to access Google Cloud services at the project's expense. The Firebase project configuration (project ID, storage bucket, sender ID) provides a complete map of the backend infrastructure that could be used for targeted attacks against the Firebase project.

3. Network Security and Communication

3.1 SSL Hostname Verification Bypass (CRITICAL)

The application includes a method **ReactNativeBlobUtilUtils.getUnsafeOkHttpClient()** that creates an OkHttpClient instance which accepts ALL SSL hostnames by returning true from the HostnameVerifier.verify() callback. This completely bypasses SSL hostname verification, enabling man-in-the-middle (MITM) attacks on any connection that uses this HTTP client. An attacker on the same network (e.g., public Wi-Fi) could intercept and modify all traffic between the app and its backend servers, including authentication tokens, user data, and API requests.

File: com/ReactNativeBlobUtil/ReactNativeBlobUtilUtils.java

```
newBuilder.hostnameVerifier(new HostnameVerifier() { public boolean verify(String str, SSLSession
sSLSession) { return true; } });
```

3.2 Missing Network Security Configuration (HIGH)

The application does not define a **network_security_config.xml** file, and the AndroidManifest.xml does not reference the android:networkSecurityConfig attribute. This means the app relies entirely on the platform's default security policy. While Android 7+ blocks cleartext traffic by default, there is no explicit enforcement, no certificate pinning definitions, and no custom trust anchors configured. This leaves the app vulnerable to future misconfigurations that could allow cleartext traffic, and provides no defense-in-depth against TLS downgrade or MITM attacks using compromised certificate authorities.

3.3 No Certificate Pinning (MEDIUM)

The application does not implement certificate pinning for any of its API endpoints. While Expo includes a root certificate (expo-root.pem) for code signing verification of OTA updates, there is no network-level certificate pinning for the app's API communications with app.willma.life or any other backend service. This means that an attacker who can obtain a certificate from a trusted CA (through compromise, social engineering, or a state-level actor) can intercept all HTTPS traffic without detection. Certificate pinning would ensure that only certificates with the expected public key are accepted, even if a CA is compromised.

3.4 HTTP URLs in Production Build (MEDIUM)

The React Native dev support infrastructure in the production build contains multiple HTTP (non-HTTPS) URLs used for Metro bundler communication, including URLs like http://%s/status and http://%s/launch-js-devtools. While these are only active during development, their presence in the production build indicates incomplete build optimization. If the dev mode were accidentally enabled in production, all Metro traffic would be sent over cleartext, allowing MITM attacks that could inject arbitrary JavaScript code into the running application.

4. Exported Components Analysis

4.1 CropImageActivity Exported Without Protection (HIGH)

The activity **com.canhub.cropper.CropImageActivity** is declared with android:exported="true" and no intent-filter restrictions or permission guards. Any application on the device can launch this activity with crafted intents, potentially causing unexpected behavior, accessing image data being cropped, or crashing the app through malformed input. This is a common vulnerability in Android apps where third-party library activities are exported by default. The fix is straightforward: set android:exported="false" for this activity in the manifest, or add a custom signature-level permission to restrict access to apps signed with the same key.

4.2 Other Exported Components

Component	Exported	Risk
CustomTabActivity (Facebook)	true	LOW - Standard FB SDK OAuth flow
ReactNativeFirebaseMessagingReceiver	true (with perm)	INFO - Protected by C2DM permission
FirebaseInstanceIdReceiver	true (with perm)	INFO - Protected by C2DM permission
DiagnosticsReceiver (WorkManager)	true (with DUMP perm)	LOW - System-only permission
ProfileInstallReceiver	true (with DUMP perm)	LOW - System-only permission

SystemJobService (WorkManager)	true (with BIND_JOB_SERVICE)	INFO - Standard AndroidX component

5. WebView Security

5.1 Apple Sign-In WebView JavaScript Interface (HIGH)

The Apple Sign-In WebView (SignInWebViewDialogFragment) enables JavaScript and uses addJavascriptInterface to expose a FormInterceptorInterface to the WebView. While the minSdkVersion of 23 eliminates the well-known reflection-based attack vector on pre-API 17 devices, the exposed JavaScript interface could still be exploited if the Apple authentication page is compromised or redirected through phishing. The interface methods could be called by any JavaScript executing within the WebView, potentially leaking authentication state data or interfering with the sign-in flow. The WebView also enables setJavaScriptCanOpenWindowsAutomatically, which could allow popup-based attacks.

5.2 React Native WebView Bridge (MEDIUM)

The RNCWebView component adds a RNCWebViewBridge JavaScript interface when messaging is enabled. This is the standard React Native WebView bridge mechanism, but it expands the attack surface of any WebView that loads external content. Any web page loaded in the WebView could call methods on the bridge object. If the WebView loads arbitrary or user-controlled URLs, this becomes a significant risk, as malicious JavaScript could interact with the native layer through the bridge. On the positive side, the RNCWebViewManagerImpl correctly sets secure defaults including setAllowFileAccess(false), setAllowContentAccess(false), and setAllowFileAccessFromFileURLs(false), which prevents local file access through WebViews.

6. Insecure Data Storage

6.1 android:allowBackup Enabled (HIGH)

The application has android:allowBackup="true" set in the manifest. This allows the app's data to be backed up via ADB (Android Debug Bridge), including SharedPreferences, SQLite databases, and internal files. This data may contain authentication tokens, user preferences, cached credentials, and other sensitive information. Anyone with physical access to the device (or via ADB over USB debugging) can extract the app's complete data directory using the command "adb backup com.willma.client". This backup can then be parsed to extract all stored data. The fix is to set android:allowBackup="false" and optionally specify android:fullBackupContent to explicitly control which data, if any, can be included in auto-backups.

6.2 Facebook Client Token in Plaintext Resources (CRITICAL)

As detailed in Section 2, the Facebook Client Token is hardcoded in strings.xml and referenced as a meta-data value in the AndroidManifest. This token is trivially extractable from the APK by anyone using tools like apktool, jadx, or even a simple ZIP extraction. The token can then be used to make API calls on behalf of the Facebook application integration, potentially accessing user data or performing actions as the app. This is particularly dangerous because the Facebook SDK Auto-Init and Auto-Log App Events are both enabled (set to true in the manifest), meaning the SDK initializes and begins tracking as soon as the app starts, using this exposed token.

7. Debug and Backup Flags

7.1 Development Support Code in Release Build (MEDIUM)

The production build contains numerous React Native development support artifacts. The file rn_dev_preferences.xml includes settings for JS Dev Mode, Minify, and Debug Server Host configuration. The strings.xml file contains the text "DEVELOPMENT CLIENT" as a splash screen string. While the DevSettingsActivity itself is marked exported="false", the presence of development preferences and strings in a release build indicates incomplete build optimization and could leak information about the development setup. More critically, if the dev menu can be triggered through the Expo gesture (shaking the device or three-finger long press), it could expose debugging capabilities including remote JS debugging, performance monitoring, and element inspection.

7.2 Expo Dev Menu Components Present (LOW)

The Expo dev menu module (expo.modules.devmenu) is included in the release build. The DevMenuDevToolsDelegate class exists with methods like openJSInspector, toggleRemoteDebugging, reload, and togglePerformanceMonitor. While these are typically disabled in production builds, their presence in the binary increases the attack surface. A determined attacker could potentially enable these features through runtime manipulation or hooking frameworks like Frida or Xposed. The recommended action is to verify that the dev menu is properly disabled in the release build and that the DevMenuInternalSettingsWrapper has all debug features disabled by default.

8. Input Validation and File Provider Issues

8.1 Overly Broad FileProvider Paths (MEDIUM)

Multiple FileProvider path configurations use overly broad path definitions, exposing entire directory trees rather than specific subdirectories. The provider_paths.xml defines external-path with path=".", which exposes the entire external storage directory. The file_viewer_provider_paths.xml defines files-path with path="/" and cache-path with path="/", exposing the complete internal files and cache directories. The file_provider_paths.xml similarly uses external-path with path=".". These broad path definitions mean that any application receiving a content URI from WILLMA could access any file within these directories, potentially including cached authentication tokens, user data, or downloaded content that was meant to be private.

File	Path Definition	Exposure
provider_paths.xml	external-path path="."	All external storage
file_viewer_provider_paths.xml	files-path path="/" + cache-path path="/" + external-path path="."	All internal + external
file_provider_paths.xml	external-path path="."	All external storage
ivpusic_imagepicker_provider_paths.xml	external-path path="."	All external storage

9. Permission Analysis

The application requests a total of 49 permissions, many of which are concerning from a security and privacy perspective. Below is an analysis of the most problematic permissions that could be exploited by users or could raise red flags during Google Play review.

Permission	Risk Level	Concern
READ_PHONE_STATE	MEDIUM	Access to IMEI, phone number, call state - unnecessary for most apps
SYSTEM_ALERT_WINDOW	LOW	Allows overlay on other apps - potential tapjacking risk
DOWNLOAD_WITHOUT_NOTIFICATION	LOW	Silent downloads - suspicious behavior signal
ACCESS_FINE_LOCATION	INFO	Precise GPS location - ensure proper justification
ACCESS_COARSE_LOCATION	INFO	Approximate location - ensure proper justification
READ_CALENDAR / WRITE_CALENDAR	INFO	Calendar access - ensure proper justification
RECORD_AUDIO	INFO	Microphone access - ensure proper justification
CAMERA	INFO	Camera access - ensure proper justification
READ_EXTERNAL_STORAGE	INFO	File access - standard but review scope
WRITE_EXTERNAL_STORAGE	INFO	File write - restricted to SDK < 30
ACTIVITY_RECOGNITION	LOW	Physical activity detection - unclear purpose

The READ_PHONE_STATE permission is particularly concerning as it provides access to device identifiers (IMEI/MEID) and call state, which is rarely justified for a conversational AI application. Google Play has increasingly rejected apps that request this permission without a clear and compelling use case. The SYSTEM_ALERT_WINDOW permission allows the app to draw over other applications, which could be used for tapjacking attacks if an attacker can inject content into the app. The DOWNLOAD_WITHOUT_NOTIFICATION permission is a system-level permission that is typically only granted to system apps and signals potentially stealthy behavior to both users and Google Play reviewers.

10. React Native / Expo Specific Issues

10.1 Expo OTA Update Security (MEDIUM)

The application has Expo Updates enabled with EXPO_UPDATES_CHECK_ON_LAUNCH set to ALWAYS and a 300-second launch wait time (5 minutes). The update URL points to https://u.expo.dev/0c01da2c-5409-459b-a106-98322e3d7076 on the production channel. While Expo includes code signing verification capabilities (CodeSigningConfiguration is present in the codebase and an expo-root.pem certificate is bundled), the critical question is whether code signing is actually enforced or if allowUnsignedManifests is set to true. If code signing is not enforced, an attacker who can perform a MITM attack on the update connection (especially given the lack of certificate pinning) could push malicious JavaScript updates that would execute in the app context with full access to native capabilities including the camera, microphone, location, and all stored data.

10.2 Microsoft Clarity Analytics (INFO)

The application includes the Microsoft Clarity analytics SDK (com.microsoft.clarity), which tracks user sessions including WebView content and DOM mutations. The clarity.js file is present in the assets directory. This tracking

SDK captures detailed user interaction data that may include sensitive information entered in WebViews, such as personal messages, health data, or financial information shared with the AI assistant. Users should be clearly informed about this tracking through a privacy policy, and the app must comply with applicable data protection regulations including GDPR, CCPA, and any regional privacy laws. The inclusion of this SDK without clear disclosure could result in Google Play policy violations and potential legal liability.

10.3 Pairip License Verification (INFO)

The app includes Pairip license verification (com.pairip.licensecheck), which is Google Play's anti-piracy mechanism. The LicenseActivity and LicenseContentProvider components are properly marked as exported="false". The RSA public key for license verification is hardcoded in LicenseClient.java. While this is a public key (not a private key) and is designed to be distributed, it can be used to analyze and potentially bypass the license verification system. However, this is a standard component and the implementation follows Google's recommended practices.

11. Complete Findings Summary

#	Severity	Category	Finding	File
1	CRITICAL	Credential Leak	OpenAI API Key hardcoded in app.config	app.config
2	CRITICAL	Credential Leak	Anthropic API Key hardcoded in app.config	app.config
3	CRITICAL	Credential Leak	DeepSeek API Key hardcoded in app.config	app.config
4	CRITICAL	Credential Leak	Gemini API Key hardcoded in app.config	app.config
5	CRITICAL	SSL Bypass	HostnameVerifier accepts all hostnames	ReactNativeBlobUtilUtils.java
6	CRITICAL	Credential Leak	Facebook Client Token in plaintext	strings.xml
7	HIGH	Credential Leak	Google/Firebase API Key hardcoded	strings.xml
8	HIGH	Credential Leak	Facebook App ID and Client Token exposed	strings.xml
9	HIGH	Network Security	Missing network_security_config.xml	AndroidManifest.xml
10	HIGH	Exported Component	CropImageActivity exported without protection	AndroidManifest.xml
11	HIGH	WebView Security	Apple Sign-In WebView JS interface	SignInWebViewDialogFragment.java
12	HIGH	Data Exposure	android:allowBackup=true enabled	AndroidManifest.xml
13	HIGH	Credential Leak	Ramadan API Key hardcoded	app.config
14	HIGH	Credential Leak	Sentry DSN exposed	app.config
15	MEDIUM	Insecure Communication	No certificate pinning implemented	N/A
16	MEDIUM	Insecure Communication	HTTP URLs in production build	DevServerHelper.java
17	MEDIUM	WebView Security	RN WebView addJavascriptInterface	RNCWebView.java
18	MEDIUM	Credential Leak	Firebase config IDs exposed	strings.xml
19	MEDIUM	Debug Info	Dev support strings in release build	rn_dev_preferences.xml

#	Severity	Category	Finding	File
1	MEDIUM	Path Traversal	Overly broad FileProvider paths	Multiple XML files
2	MEDIUM	Excessive Permissions	READ_PHONE_STATE permission	AndroidManifest.xml
3	MEDIUM	Update Security	Expo Updates - verify code signing	AndroidManifest.xml
4	MEDIUM	Credential Leak	GraphQL API URL exposed	app.config
5	MEDIUM	Tracking	Facebook SDK auto-tracking enabled	AndroidManifest.xml
6	MEDIUM	Credential Leak	Expo Project ID exposed	app.config
7	MEDIUM	Credential Leak	License RSA Public Key exposed	LicenseClient.java
8	LOW	Exported Component	CustomTabActivity exported (FB SDK)	AndroidManifest.xml
9	LOW	Excessive Permissions	SYSTEM_ALERT_WINDOW permission	AndroidManifest.xml
10	LOW	Suspicious Permission	DOWNLOAD_WITHOUT_NOTIFICATION	AndroidManifest.xml
11	LOW	Debug Exposure	Dev menu components in release	DevMenuDevToolsDelegate.java
12	LOW	Information Disclosure	Facebook App ID exposed	strings.xml
13	LOW	Excessive Permissions	ACTIVITY_RECOGNITION permission	AndroidManifest.xml
14	LOW	Credential Leak	Sentry verification token	verification.properties
15	INFO	Exported Component	FCM receivers (standard config)	AndroidManifest.xml
16	INFO	WebView Security	WebView secure defaults (positive)	RNCWebViewManagerImpl.java
17	INFO	Tracking	Microsoft Clarity analytics included	clarity.js
18	INFO	License	Pairip license verification (standard)	LicenseClient.java

12. Priority Remediation Plan

12.1 Immediate Actions (Before Google Play Launch)

The following actions are **mandatory** and must be completed before publishing the application to the Google Play Store. Failure to address these issues will expose the application to significant financial risk, user data compromise, and potential Google Play policy violations.

- â■¢ **Rotate ALL LLM API keys immediately** - The OpenAI, Anthropic, DeepSeek, and Gemini API keys are compromised and must be rotated. Generate new keys and do NOT embed them in the client-side code.
- â■¢ **Move API keys to a secure backend** - All AI API calls should be proxied through your own backend server (e.g., the GraphQL API at app.willma.life). The client should never directly communicate with LLM providers.
- â■¢ **Remove getUnsafeOkHttpClient()** - Delete or strictly gate behind build configuration the method that bypasses SSL hostname verification. This is a critical security vulnerability.
- â■¢ **Rotate the Facebook Client Token** - The current token is exposed. Generate a new one and use the Facebook SDK's built-in token handling instead of hardcoding it.
- â■¢ **Set android:allowBackup="false"** - Prevent extraction of app data via ADB backup. Also add android:fullBackupContent to explicitly control backup scope.

â■¢ **Set CropImageActivity exported="false"** - Prevent unauthorized apps from launching this activity.

12.2 Short-term Actions (Within 1 Week)

â■¢ **Add network_security_config.xml** - Explicitly block cleartext traffic, define certificate pinning for API endpoints, and configure custom trust anchors.

â■¢ **Implement certificate pinning** - Use OkHttp CertificatePinner or Android Network Security Configuration to pin certificates for app.willma.life and other backend services.

â■¢ **Restrict Google API key** - In Google Cloud Console, restrict the API key to only the app's package name and SHA-1 fingerprint.

â■¢ **Narrow FileProvider paths** - Replace path="." and path="/" with specific subdirectories that need to be shared.

â■¢ **Remove READ_PHONE_STATE permission** - This permission is unnecessary for an AI assistant app and will trigger Google Play review concerns.

â■¢ **Verify Expo code signing enforcement** - Ensure CODE_SIGNING_ENABLED is true and unsigned manifests are rejected.

12.3 Medium-term Actions (Within 1 Month)

â■¢ **Strip dev support code from release builds** - Configure ProGuard/R8 rules to remove React Native dev support code and strings from production builds.

â■¢ **Review and reduce permissions** - Audit all requested permissions and remove any that are not essential to the app's core functionality.

â■¢ **Add API rate limiting** - Implement rate limiting on all API endpoints to mitigate damage from any future key leakage.

â■¢ **Implement API key restrictions** - On all AI provider consoles, set up usage limits, allowed IP ranges, and billing alerts.

â■¢ **Review Microsoft Clarity integration** - Ensure compliance with privacy regulations and add clear disclosure in the privacy policy.

â■¢ **Remove DOWNLOAD_WITHOUT_NOTIFICATION** - This system-level permission will not be granted and signals suspicious behavior.